

This file is a merged representation of a subset of the codebase, containing specifically included files, combined into a single document by Repomix.

<file_summary>

This section contains a summary of **this** file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems **for** analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (**if** enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not **this** packed version.
- When processing **this** file, use the file path to distinguish between different files in the repository.
- Be aware that **this** file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in **this** packed representation. Please refer to the Repository Structure section **for** a complete list of file paths, including binary files
- Only files matching these patterns are included: ****/*.cs**
- *Files matching patterns in .gitignore are excluded*
- *Files matching default ignore patterns are excluded*
- *Files are sorted by Git change count (files with more changes are at the bottom)*

</notes>

</file_summary>

<directory_structure>

```
Assets/
  _Scripts/
    Brains/
      EnemyBrain.cs
    Enums/
      GameEnums.cs
    Interfaces/
      IEstado.cs
    Player/
      PlayerController.cs
```

```
PlayerFear.cs
PlayerNoise.cs
States/
  EstadoRastreo.cs
Systems/
  IntuicionSystem.cs
  TrampaAcustica.cs
UI/
  UI_BarraMiedo.cs
  UI_MenuManager.cs
Input/
  PlayerControls.cs
UI_Inicio/
  MenuInicioController.cs
</directory_structure>
```

<files>

This section contains the contents of the repository's files.

<file path="Assets/_Scripts/Player/PlayerFear.cs">

```
// -----
// PlayerFear.cs
// Se suscribe a la "radio" del IntuicionSystem y ajusta el movimiento del
// jugador basándose en los niveles de miedo del enemigo.
// Este script DEBE estar adjunto al mismo GameObject que tiene el PlayerController
// (por ejemplo, la cápsula del jugador).
// -----
using UnityEngine;

namespace _Scripts.Player
{
    // Garantiza de forma segura que el GameObject tenga un PlayerController.
    // Si no existe, Unity lo añadirá automáticamente para evitar errores de
    // referencia nula en tiempo de ejecución.
    [RequireComponent(typeof(PlayerController))]
    public class PlayerFear : MonoBehaviour
    {
        [Header("Intuicion System (Data Asset)")]
        [Tooltip("Asigna aquí el ScriptableObject de IntuicionSystem que utiliza el EnemyBrain.")]
        [SerializeField] private _Scripts.Systems.IntuicionSystem intuicionSystem;

        // Referencia interna al controlador de movimiento del jugador.
        private PlayerController playerController;

        // Nivel de miedo actual del jugador. Rango: 0 (calma total) hasta 4 (terror absoluto).
        private int fearLevel = 0;

        // LÃmite mÃximo del nivel de miedo. Corresponde a un factor de 0.8 (80% de penalizaciÃn de velocidad).
        private const int MaxFearLevel = 4;

        /// <summary>
        /// Awake se ejecuta al cargar la instancia del script, antes de que empiece el juego.
```

```

    /// Ideal para inicializar y capturar componentes internos de forma op
    timizada sin usar GameObject.Find.
    /// </summary>
    private void Awake()
    {
        // Cacheamos el componente PlayerController que estÃ; en este mism
o objeto.
        playerController = GetComponent<PlayerController>();
    }

    /// <summary>
    /// Se activa automÃ;ticamente cada vez que el objeto se habilita en l
a escena.
    /// AquÃ- nos "sintonizamos" a la radio del sistema de intuiciÃ³n (Eve
ntos).
    /// </summary>
    private void OnEnable()
    {
        // VerificaciÃ³n de seguridad por si olvidaste arrastrar el Script
ableObject en el Inspector.
        if (intuicionSystem != null)
        {
            // Nos suscribimos a los eventos. Cuando la IA suba o baje de
fase,
            // este script se enterarÃ; inmediatamente de forma asÃ-ncrona
.
            intuicionSystem.OnSubirFase += HandleSubirFase;
            intuicionSystem.OnBajarFase += HandleBajarFase;
        }
    }

    /// <summary>
    /// Se activa automÃ;ticamente cuando el objeto se deshabilita o se de
struye.
    /// CRUCIAL: Siempre hay que desvincular los eventos para evitar fugas
de memoria (Memory Leaks).
    /// </summary>
    private void OnDisable()
    {
        if (intuicionSystem != null)
        {
            // Nos desuscribimos de los eventos para limpiar las referenci
as en memoria.
            intuicionSystem.OnSubirFase -= HandleSubirFase;
            intuicionSystem.OnBajarFase -= HandleBajarFase;
        }
    }

    /// <summary>
    /// MÃ©todo manejador que se ejecuta automÃ;ticamente cuando el sistem
a avisa que el enemigo subiÃ³ de fase.
    /// </summary>
    private void HandleSubirFase()
    {
        // Incrementamos el miedo en 1, pero usamos Mathf.Clamp para asegu
rar
        // que matemÃ;ticamente nunca supere el MaxFearLevel (4) ni baje d
e 0.
        fearLevel = Mathf.Clamp(fearLevel + 1, 0, MaxFearLevel);

```

```

    // Recalculamos y aplicamos el peso del terror al jugador.
    ApplyFearPenalty();
}

    /// <summary>
    /// MÃ©todo manejador que se ejecuta automÃ;ticamente cuando el sistem
a avisa que el enemigo bajÃ³ de fase (se enfriÃ³).
    /// </summary>
    private void HandleBajarFase()
    {
        // Decrementamos el miedo en 1, manteniendo el valor protegido ent
re 0 y 4.
        fearLevel = Mathf.Clamp(fearLevel - 1, 0, MaxFearLevel);

        // Recalculamos y aplicamos el peso del terror al jugador.
        ApplyFearPenalty();
    }

    /// <summary>
    /// Traduce el nivel de miedo entero (0 a 4) en un porcentaje decimal
de penalizaciÃ³n fÃ-sica
    /// y se lo inyecta al PlayerController.
    /// </summary>
    private void ApplyFearPenalty()
    {
        // Control de daÃ±os: Si por alguna razÃ³n el script se rompiÃ³ o
no tiene el controlador, cancela la ejecuciÃ³n.
        if (playerController == null) return;

        // EXPLICACIÃ³N MATEMÃTICA DEL FACTOR:
        // 1. (float)fearLevel de la lÃnea inferior convierte el entero a
float para permitir divisiones decimales exactas.
        // 2. Se divide entre MaxFearLevel (4) para obtener un porcentaje
normalizado de 0.0 a 1.0.
        // 3. Se multiplica por 0.8f para escalar ese porcentaje a un rang
o final de 0.0 a 0.8.
        // Ejemplos de resultado segÃ³n la fase:
        // - Miedo 0: (0 / 4) * 0.8 = 0.0 -> 0% de reducciÃ³n (Velocidad
normal).
        // - Miedo 1: (1 / 4) * 0.8 = 0.2 -> 20% de reducciÃ³n.
        // - Miedo 2: (2 / 4) * 0.8 = 0.4 -> 40% de reducciÃ³n.
        // - Miedo 3: (3 / 4) * 0.8 = 0.6 -> 60% de reducciÃ³n.
        // - Miedo 4: (4 / 4) * 0.8 = 0.8 -> 80% de reducciÃ³n (El jugado
r apenas puede arrastrarse del susto).
        float factor = (float)fearLevel / MaxFearLevel * 0.8f;

        // Le enviamos este factor limpio al PlayerController para que res
te velocidad o altere el movimiento.
        playerController.AplicarPenalizacionMiedo(factor);
    }
}

</file>

<file path="Assets/_Scripts/Player/PlayerNoise.cs">
using UnityEngine;
using _Scripts.Systems; // Para ver el IntuicionSystem
using _Scripts.Enums;   // Para ver el PerfilAcustico

namespace _Scripts.Player

```

```

{
    /**
     * CLASE: PlayerNoise (El Generador de Pulsos)
     * Se encarga de medir el tiempo entre pasos, reproducir el audio
     * local para asustar al jugador humano, y enviar el "Paquete de Datos"
     * matemático al cerebro del enemigo.
     */
}

public class PlayerNoise : MonoBehaviour
{
    [Header("Comunicaciones (Los Cables)")]
    [Tooltip("El 'buzón' central donde dejaremos el paquete de ruido")]
    /*Oye, en este espacio solo se pueden guardar cosas que tengan la estructura de IntuicionSystem
    dataIntuicion es solo un puntero inalambrico*/
    [SerializeField] private IntuicionSystem dataIntuicion;

    [Tooltip("El cuerpo del enemigo para poder calcular a cuántos metros estamos")]
    [SerializeField] private Transform transformEnemigo;

    [Header("Configuración de Audio Físico")]
    [Tooltip("El parlante que reproducirá los .wav en los pies/boca del jugador")]
    [SerializeField] private AudioSource emisorDeSonido;

    [Tooltip("Audios para cuando el jugador va lento/agachado")]
    [SerializeField] private AudioClip audioSusurro;

    [Tooltip("Audios para cuando el jugador camina normal")]
    [SerializeField] private AudioClip audioPasoNormal;

    [Tooltip("Audios para cuando el jugador corre asustado")]
    [SerializeField] private AudioClip audioGritoSusto;

    // --- EL ETIQUETADOR DE ESTADOS ---
    // Este enum local nos dice qué está; haciendo el jugador AHORA MISMO
    // Más adelante, el script de movimiento cambiará; esta variable automáticamente.
    public enum EstadoMovimiento { Quieto, Agachado, Caminando, Corriendo }

    [Header("Estado Actual (Pruebas)")]
    [Tooltip("Cambia esto en Unity mientras juegas para probar los distintos ruidos")]
    public EstadoMovimiento estadoActual = EstadoMovimiento.Quieto;

    // El segundero que cuenta el tiempo entre un paso y otro
    private float _cronometroPulso = 0f;

    private void Start()
    {
        // Cláusulas de salvaguarda: Avisamos si olvidaste conectar algo en el Inspector
        if (!dataIntuicion) Debug.LogError("PlayerNoise: Falta conectar el IntuicionSystem.");
        if (!transformEnemigo) Debug.LogError("PlayerNoise: Falta el Transform del Enemigo.");
        if (!emisorDeSonido) Debug.LogError("PlayerNoise: Falta el AudioSource.");
    }
}

```

```

    }

    private void Update()
    {
        // Si falta el buzÃ³n de datos, no hacemos nada, para, evitar que
        el juego explote
        if (!dataIntuicion || !transformEnemigo) return;

        // 1. EL RELOJ: Definimos quÃ© tan rÃ¡pido late el pulso segÃºn el
        estado
        float tiempoEntrePulsos = 0f;

        switch (estadoActual)
        {
            case EstadoMovimiento.Quieto:
                _cronometroPulso = 0f; // Reseteamos el reloj
                return; // Salimos del Update, no hay ruido que hacer

            case EstadoMovimiento.Agachado:
                tiempoEntrePulsos = 1.2f; // Un pulso lento (mucho espacio
                entre pasos)

                break;

            case EstadoMovimiento.Caminando:
                tiempoEntrePulsos = 0.7f; // Un pulso normal

                break;

            case EstadoMovimiento.Corriendo:
                tiempoEntrePulsos = 0.3f; // Un pulso rapidÃ­simo (pasos a
                celerados)

                break;
        }

        // 2. EL SEGUNDERO AVANZA
        _cronometroPulso += Time.deltaTime;

        // 3. EL DISPARO
        if (_cronometroPulso >= tiempoEntrePulsos)
        {
            EmitirRuidoDelPulso();
            _cronometroPulso = 0f; // Vaciamos el reloj para empezar a con
            tar el siguiente paso
        }
    }

    /* * *
    * MÃTODO: EmitirRuidoDelPulso
    * Se ejecuta solo en el frame exacto en el que el pie toca el suelo.
    * Prepara el paquete, reproduce el .wav y lo envÃ­a por correo.
    */
    // ReSharper disable Unity.PerformanceAnalysis
    private void EmitirRuidoDelPulso()
    {
        // A. Calculamos la distancia real en metros usando trigonometrÃ­a
        de Unity
        float distanciaAlEnemigo = Vector3.Distance(transform.position, tr
        ansformEnemigo.position);

        // B. Preparamos las variables que meteremos en el paquete
        float volumenPaquete = 0f;
    }
}

```

```

        PerfilAcustico perfilPaquete = PerfilAcustico.GraveFisico; // Por defecto
        AudioClip clipFalsoParaHumano = null;

        // C. Rellenamos el paquete según el estado
        switch (estadoActual)
        {
            case EstadoMovimiento.Agachado:
                volumenPaquete = 0.1f; // Muy bajito
                perfilPaquete = PerfilAcustico.AgudoVocal; // Un susurro que se apaga rápidamente en la distancia
                clipFalsoParaHumano = audioSusurro;
                break;

            case EstadoMovimiento.Caminando:
                volumenPaquete = 0.3f; // Medio
                perfilPaquete = PerfilAcustico.GraveFisico; // Un paso que hace vibrar el suelo
                clipFalsoParaHumano = audioPasoNormal;
                break;

            case EstadoMovimiento.Corriendo:
                volumenPaquete = 0.8f; // Escandaloso
                perfilPaquete = PerfilAcustico.AgudoVocal; // Un grito/jadeo de pánico
                clipFalsoParaHumano = audioGritoSusto;
                break;
        }

        // D. LA MAGIA: Actualizamos la posición sospechosa...
        dataIntuicion.posicionSospechosa = transform.position;

        // ... Y enviamos el paquete matemático al IntuicionSystem
        // ¡Fíjate cómo usamos el Enum aquí! como una etiqueta de envío!
        dataIntuicion.ModificarIntuicion(volumenPaquete, distanciaAlEnemigo, perfilPaquete);

        // E. EL TEATRO: Reproducimos el archivo .wav real para asustar al jugador
        if (emisorDeSonido && clipFalsoParaHumano)
        {
            emisorDeSonido.PlayOneShot(clipFalsoParaHumano);
        }

        // Un log para que tú como desarrollador veas que el pulso salió bien
        Debug.Log($"<color=yellow>PULSO: {estadoActual} | Vol: {volumenPaquete} | Perfil: {perfilPaquete} | Dist: {distanciaAlEnemigo:F1}m</color>");
    }
}
</file>

<file path="Assets/_Scripts/Systems/TrampaAcustica.cs">
using UnityEngine;
using System.Collections;
using _Scripts.Enums;

namespace _Scripts.Systems

```

```

[RequireComponent(typeof(AudioSource))]
public class TrampaAcustica : MonoBehaviour
{
    public IntuicionSystem dataIntuicion;
    [Range(0f, 1f)] public float volumenAlerta = 0.3f;
    public PerfilAcustico perfilAcustico = PerfilAcustico.GraveFisico;
    public float tiempoApagado = 4.0f;

    private AudioSource _audioSource;
    private bool _jugadorPresente = false;
    private Coroutine _rutinaApagado;

    private void Awake()
    {
        _audioSource = GetComponent<AudioSource>();
        _audioSource.playOnAwake = false;
        _audioSource.loop = true;
        _audioSource.spatialBlend = 1.0f;
    }

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            _jugadorPresente = true;

            if (_rutinaApagado != null)
            {
                StopCoroutine(_rutinaApagado);
                _rutinaApagado = null;
            }

            if (!_audioSource.isPlaying)
            {
                _audioSource.Play();
            }

            if (dataIntuicion != null)
            {
                dataIntuicion.posicionSospechosa = transform.position;

                GameObject enemy = GameObject.FindGameObjectWithTag("Enemy");

                if (enemy != null)
                {
                    float distanciaAlMonstruo = Vector3.Distance(transform.position, enemy.transform.position);
                    dataIntuicion.ModificarIntuicion(volumenAlerta, distanciaAlMonstruo, perfilAcustico);
                }
            }
        }
    }

    private void OnTriggerExit(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            _jugadorPresente = false;

```

```

        if (_rutinaApagado != null)
        {
            StopCoroutine(_rutinaApagado);
        }
        _rutinaApagado = StartCoroutine(RutinaApagarAudio());
    }

    private IEnumerator RutinaApagarAudio()
    {
        yield return new WaitForSeconds(tiempoApagado);

        if (!_jugadorPresente && _audioSource.isPlaying)
        {
            _audioSource.Stop();
        }
        _rutinaApagado = null;
    }
}
</file>

<file path="Assets/_Scripts/UI/UI_BarraMiedo.cs">
using UnityEngine;
using UnityEngine.UI;
using _Scripts.Systems;

namespace _Scripts.UI
{
    public class UI_BarraMiedo : MonoBehaviour
    {
        [Header("Intuicion System (Data Asset)")]
        [Tooltip("Asigna aquí el ScriptableObject de IntuicionSystem")]
        [SerializeField] private IntuicionSystem intuicionSystem;

        [Header("UI Elementos")]
        [SerializeField] private Image[] diamantesUI;

        // El nivel base de miedo en el EnemyBrain es 1.
        private int nivelMiedo = 1;

        private void OnEnable()
        {
            if (intuicionSystem != null)
            {
                intuicionSystem.OnSubirFase += HandleSubirFase;
                intuicionSystem.OnBajarFase += HandleBajarFase;
            }
            ActualizarUI();
        }

        private void OnDisable()
        {
            if (intuicionSystem != null)
            {
                intuicionSystem.OnSubirFase -= HandleSubirFase;
                intuicionSystem.OnBajarFase -= HandleBajarFase;
            }
        }
    }
}

```

```

    private void HandleSubirFase()
    {
        nivelMiedo = Mathf.Clamp(nivelMiedo + 1, 1, 4);
        ActualizarUI();
    }

    private void HandleBajarFase()
    {
        nivelMiedo = Mathf.Clamp(nivelMiedo - 1, 1, 4);
        ActualizarUI();
    }

    private void ActualizarUI()
    {
        if (diamantesUI == null) return;

        for (int i = 0; i < diamantesUI.Length; i++)
        {
            if (diamantesUI[i] != null)
            {
                // Nivel 1 = 1 diamante, Nivel 2 = 2 diamantes, etc.
                diamantesUI[i].enabled = (i < nivelMiedo);
            }
        }
    }
}
</file>

<file path="Assets/UI_Inicio/MenuInicioController.cs">
using UnityEngine;
using UnityEngine.UIElements;
using UnityEngine.SceneManagement;

namespace _Scripts.UI
{
    [DefaultExecutionOrder(100)]
    public class MenuInicioController : MonoBehaviour
    {
        [SerializeField]
        private string escenaGameplay = "Catacumbas";

        private Button botonJugar;

        private void OnEnable()
        {
            var root = GetComponent<UIDocument>().rootVisualElement;
            if (root != null)
            {
                botonJugar = root.Q<Button>("btnJugar");
                if (botonJugar != null)
                {
                    botonJugar.clicked += () => UnityEngine.SceneManagement.SceneManager.LoadScene("Catacumbas");
                }
                else
                {
                    Debug.LogWarning("No se encontró el elemento 'btnJugar' de tipo Button.");
                }
            }
        }
    }
}

```

```

    }
}

private void OnDisable()
{
    // Nota: Al usar una expresiÃ³n lambda anÃ³nima, no se puede desre
    gistrar directamente con '--'.
    // Sin embargo, este es el formato exacto solicitado.
}
}
}
</file>

<file path="Assets/_Scripts/Enums/GameEnums.cs">
/* * ESTE SCRIPT ES EL "DICCIONARIO" DEL JUEGO.
 * AquÃ- definimos los nombres oficiales de los estados para que
 * el cÃ³digo no tenga errores de ortografÃ-a al comunicarse.
 */

namespace _Scripts.Enums
{
    // Las fases por las que pasarÃ; el Jugador segÃ³n su miedo
    public enum FasePlayer
    {
        Alerta,        // El estado normal, explorando con cautela.
        Sobresalto,    // Cuando algo ocurre de repente (un susto leve).
        Colapso,        // El miedo es tan alto que el movimiento falla o se rale
ntiza.
        Horror          // Estado de muerte o evento final (pÃ³rdida de control).
    }

    // Las fases de comportamiento de la Inteligencia Artificial (Enemigo)
    public enum FaseEnemigo
    {
        Rastreo,        // El enemigo camina buscando pistas sin saber dÃ³nde es
tÃ;s.
        Sigilo,          // El enemigo sabe que estÃ;s cerca y se mueve sin hacer
ruido.
        Persecucion,    // El enemigo te vio y va tras de ti (mÃ;xima velocidad)
.
        SedDeSangre     // El estado potenciado donde el enemigo no se detiene p
or nada.
    }
}
</file>

<file path="Assets/_Scripts/Interfaces/IEstado.cs">
/* * ESTE SCRIPT ES EL "CONTRATO" DE LOS ESTADOS.
 * No es una clase, es una Interface.
 * Obliga a cualquier estado (Alerta, PersecuciÃ³n, etc.) a tener
 * estos tres momentos clave para que el cerebro del enemigo o jugador
 * sepa cÃ³mo activarlos.
 */

namespace _Scripts.Interfaces
{
    public interface IEstado
    {
        // Se ejecuta una sola vez al entrar al estado.
        // Ideal para: Activar animaciones, sonidos de inicio o cambiar la nie

```

```

bla.

void Entrar();

// Se ejecuta constantemente mientras el estado estÃ© activo.
// Ideal para: Calcular el movimiento o revisar si el enemigo nos ve.
void Ejecutar();

// Se ejecuta una sola vez al salir del estado.
// Ideal para: Apagar sonidos, limpiar efectos visuales o resetear var
iables.
void Salir();
}
}
</file>

<file path="Assets/_Scripts/Player/PlayerController.cs">
using UnityEngine;
using UnityEngine.InputSystem;

namespace _Scripts.Player
{
    public class PlayerController : MonoBehaviour
    {
        [Header("Conexiones Especializadas")]
        [SerializeField] private PlayerNoise scriptRuido;

        [Tooltip("Ã;Arrastra aquÃ- el objeto @Head que estÃ; dentro de tu Juga
dor!")]
        [SerializeField] private Transform headTransform; // <--- NUEVA CONEXI
Ã\223N VITAL

        [Header("ConfiguraciÃ³n de Velocidades (CalibraciÃ³n)")]
        [SerializeField] private float velocidadAgachado = 1.5f;
        [SerializeField] private float velocidadCaminando = 3.5f;
        [SerializeField] private float velocidadCorriendo = 6.0f;
        [SerializeField] private float velocidadRotacion = 10f;

        [Header("Suavizado de Movimiento")]
        [SerializeField] private float smoothTime = 0.1f;
        [SerializeField] private float rotationSmoothTime = 0.02f;

        private Vector3 _velocitySmooth;
        private Vector3 _velocitySmoothDerivative;
        private float _currentRotationSmoothVelocity;

        private float _yaw;
        private float _pitch;

        [Header("ConfiguraciÃ³n del Nuevo Input System")]
        [SerializeField] private InputActionReference moveAction;
        [SerializeField] private InputActionReference sprintAction;
        [SerializeField] private InputActionReference crouchAction;
        [SerializeField] private InputActionReference lookActionDebug;

        private bool _controlBloqueado;
        private float _velocidadActual;
        private Vector3 _direccionMovimiento;

        private void OnEnable()
        {

```

```

        if (moveAction != null) moveAction.action.Enable();
        if (sprintAction != null) sprintAction.action.Enable();
        if (crouchAction != null) crouchAction.action.Enable();
        if (lookActionDebug != null) lookActionDebug.action.Enable();
    }

    private void OnDisable()
    {
        if (moveAction != null) moveAction.action.Disable();
        if (sprintAction != null) sprintAction.action.Disable();
        if (crouchAction != null) crouchAction.action.Disable();
        if (lookActionDebug != null) lookActionDebug.action.Disable();
    }

    public void SetControlBlocked(bool blocked)
    {
        if (_controlBloqueado == blocked) return;

        _controlBloqueado = blocked;
        if (blocked)
        {
            _velocitySmooth = Vector3.zero;
            _velocitySmoothDerivative = Vector3.zero;
            _direccionMovimiento = Vector3.zero;
            _velocidadActual = 0f;
            _currentRotationSmoothVelocity = 0f;
        }
    }

    private void Start()
    {
        if (!scriptRuido) Debug.LogError("PlayerController: ¡Falta conectar el componente PlayerNoise!");
        if (!headTransform) Debug.LogError("PlayerController: ¡Falta conectar el Transform del Head!");

        _yaw = transform.eulerAngles.y;
        if (headTransform != null)
        {
            _pitch = headTransform.eulerAngles.x;
            if (_pitch > 180f) _pitch -= 360f;
        }

        // Ocultar y bloquear el cursor para apuntar correctamente
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    private void Update()
    {
        if (!scriptRuido || _controlBloqueado) return;

        // Leemos el input del mouse
        Vector2 mouseInput = lookActionDebug != null ? lookActionDebug.action.ReadValue<Vector2>() : Vector2.zero;

        // === ROTACIÓN 223N: CÁLCULO Incremental ===
        _yaw += mouseInput.x * velocidadRotacion * Time.deltaTime;
        _pitch -= mouseInput.y * velocidadRotacion * Time.deltaTime;

```

```

        // Limitamos la vista para que no te rompas el cuello (Gimbal Lock seguro)
        _pitch = Mathf.Clamp(_pitch, -89f, 89f);

        // 1. ROTACIÓN 223N DEL CUERPO (Suavizada): Solo rota en Y
        float smoothedYaw = Mathf.SmoothDampAngle(transform.eulerAngles.y, _yaw, ref _currentRotationSmoothVelocity, rotationSmoothTime);
        transform.rotation = Quaternion.Euler(0f, smoothedYaw, 0f);

        // 2. ROTACIÓN 223N DE LA CABEZA/CÁMARA (Instantánea): Rota en X e Y
        // En vez de pelear con Cinemachine, movemos el objeto que Cinemachine está siguiendo.
        if (headTransform != null)
        {
            headTransform.rotation = Quaternion.Euler(_pitch, _yaw, 0f);
        }

        // === MOVIMIENTO RELATIVO A LA CABEZA ===
        // Ahora caminamos hacia donde apunta el Head, proyectado en el suelo
        Vector3 forward = headTransform.forward;
        Vector3 right = headTransform.right;
        forward.y = 0f;
        right.y = 0f;
        forward.Normalize();
        right.Normalize();

        Vector2 inputVector = moveAction.action.ReadValue<Vector2>();
        _direccionMovimiento = (forward * inputVector.y + right * inputVector.x).normalized;

        // Máquina de estados de velocidad rápida
        if (_direccionMovimiento.sqrMagnitude == 0f)
        {
            _velocidadActual = 0f;
            scriptRuido.estadoActual = PlayerNoise.EstadoMovimiento.Quieto;
        }
        else if (sprintAction != null && sprintAction.action.IsPressed())
        {
            _velocidadActual = velocidadCorriendo;
            scriptRuido.estadoActual = PlayerNoise.EstadoMovimiento.Corriendo;
        }
        else if (crouchAction != null && crouchAction.action.IsPressed())
        {
            _velocidadActual = velocidadAgachado;
            scriptRuido.estadoActual = PlayerNoise.EstadoMovimiento.Agachado;
        }
        else
        {
            _velocidadActual = velocidadCaminando;
            scriptRuido.estadoActual = PlayerNoise.EstadoMovimiento.Caminando;
        }

        // === MOVIMIENTO FÍSICO ===
        Vector3 desiredVelocity = _direccionMovimiento * _velocidadActual;

```



```

        _velocitySmooth = Vector3.SmoothDamp(_velocitySmooth, desiredVelocity, ref _velocitySmoothDerivative, smoothTime);

        if (TryGetComponent<CharacterController>(out var controller))
        {
            Vector3 moveVector = _velocitySmooth * Time.deltaTime;
            if (!controller.isGrounded) moveVector.y -= 9.81f * Time.deltaTime;

            controller.Move(moveVector);
        }
        else
        {
            transform.Translate(_direccionMovimiento * (_velocidadActual * Time.deltaTime), Space.World);
        }
    }

    public void AplicarPenalizacionMiedo(float factor)
    {
        float velocidadBase = velocidadCaminando;
        _velocidadActual = Mathf.Lerp(velocidadBase, velocidadBase * 0.2f, factor);

        smoothTime = Mathf.Lerp(smoothTime, smoothTime + 0.3f, factor);
        rotationSmoothTime = Mathf.Lerp(rotationSmoothTime, rotationSmoothTime + 0.3f, factor);
    }
}
</file>

<file path="Assets/_Scripts/States/EstadoRastreo.cs">
using UnityEngine;
using _Scripts.Interfaces;
using _Scripts.Brains;

namespace _Scripts.States
{
    /* * ESTADO: RASTREO (FASE 1)
     * El enemigo patrulla tranquilamente entre waypoints usando NavMeshAgent.
     * Es el estado más "ciego" y lento.
     */
    public class EstadoRastreo : IEstado
    {
        private readonly EnemyBrain _brain;
        private readonly float _velocidadRastreo = 2.5f;

        private int _indexWaypointActual = 0;
        private Vector3 _ultimoDestinoAsignado;
        private bool _iniciando = true;

        // El constructor nos permite recibir la referencia del cerebro
        public EstadoRastreo(EnemyBrain brain)
        {
            _brain = brain;
        }

        public void Entrar()
        {
            Debug.Log("<color=green>Enemigo entrando en fase de RASTREO.</color>");

```

```

        _brain.ActualizarVelocidad(_velocidadRastreo);
        _ultimoDestinoAsignado = Vector3.zero;
        _iniciando = true;
    }

    public void Ejecutar()
    {
        // 1. Aseguramos que la velocidad actual del agente sea la de rastreo
        _brain.ActualizarVelocidad(_velocidadRastreo);

        Vector3 destinoObjetivo;
        bool esSospecha = _brain.Data.IntuicionActual > 0.2f;

        if (esSospecha)
        {
            // Si la intuición es alta, vamos a investigar la posición sospechosa
            destinoObjetivo = _brain.Data.posicionSospechosa;
        }
        else
        {
            // Si no, volvemos a la patrulla normal
            if (_brain.Waypoints == null || _brain.Waypoints.Length == 0)
            {
                return;
            }
            destinoObjetivo = _brain.Waypoints[_indexWaypointActual].position;
        }

        // 2. OPTIMIZACIÓN CRÍTICA: Solo llamamos a SetDestination si el destino ha cambiado.
        // Si llamamos a SetDestination en cada frame (Update), el agente recalcula la ruta infinitamente y se queda quieto.
        if (_iniciando || Vector3.Distance(_ultimoDestinoAsignado, destinoObjetivo) > 0.1f)
        {
            _brain.MoverHacia(destinoObjetivo);
            _ultimoDestinoAsignado = destinoObjetivo;
            _iniciando = false;
        }

        // 3. CAMBIO DE WAYPOINT: Si estamos patrullando y llegamos físicamente cerca del punto, avanzamos
        if (!esSospecha)
        {
            float distanciaFisica = Vector3.Distance(_brain.transform.position, destinoObjetivo);
            if (distanciaFisica < 0.8f)
            {
                _indexWaypointActual = (_indexWaypointActual + 1) % _brain.Waypoints.Length;
            }
        }
    }

    public void Exit() // Cumple la interfaz si requiere Salir en espacio
    {
        Salir();
    }
}

```



```

    public void Salir()
    {
        Debug.Log("Saliendo de RASTREO: El enemigo ha cambiado de estado.");
    }
}
}
</file>

<file path="Assets/_Scripts/Systems/IntuicionSystem.cs">
using UnityEngine;
using System; // Necesario para usar los "Eventos" (La Radio)

/* * * * * *
 * DICCIONARIO ACÃ232STICO
 * Lo definimos aquÃ- para que cualquier objeto del juego pueda
 * "etiquetar" su ruido antes de enviarlo al sistema.
 * * * * * */
namespace _Scripts.Enums
{
    public enum PerfilAcustico
    {
        AgudoVocal, // Gritos, susurros, cristales rotos (Se ahogan rÃ;rido co
n la distancia)
        GraveFisico // Pasos pesados, muebles cayendo (Viajan lejos por el sue
lo/estructura)
    }
}

/* * * * * *
 * EL CORAZÃ223N DE DATOS (SISTEMA DE INTUICIÃ223N)
 * ActÃªa como un Analizador de Frecuencias BÃ;sico. Recibe paquetes
 * de sonido, calcula su impacto real y alerta al enemigo.
 * * * * * */
namespace _Scripts.Systems
{
    [CreateAssetMenu(fileName = "NuevaIntuicion", menuName = "Sistema/Intuicio
n Data")]
    public class IntuicionSystem : ScriptableObject
    {
        [Header("Datos de la Fase (Escala 0.0 a 1.0)")]
        [Tooltip("El medidor de peligro de la fase actual")]
        [SerializeField] private float intuicionActual = 0f;
        public float IntuicionActual => intuicionActual;

        [Header("Memoria de PosiciÃ³n")]
        [Tooltip("Ã232ltima posiciÃ³n global donde se originÃ³ un ruido vÃ;li
do")]
        public Vector3 posicionSospechosa;

        // EVENTOS ("La Radio"): Avisan al enemigo cuando debe cambiar de fase
        public event Action OnSubirFase;
        public event Action OnBajarFase;

        /* * *
         * MÃ211TODO PARA SUBIR: LA PUERTA DE ENTRADA DEL RUIDO
         * Recibe el "Paquete de Sonido" (Volumen, Distancia y Tipo) y
         * decide cuÃ;nto asusta realmente al monstruo.
         */
        // ReSharper disable Unity.PerformanceAnalysis

```

```

        public void ModificarIntuicion(float volumenOriginal, float distanciaA
lMonstruo, _Scripts.Enums.PerfilAcustico perfil)
        {
            // Esta variable guardará; el ruido final despuÃs de calcular la
distancia
            float incrementoFinal = 0f;

            // 1. FILTRO MATEMÃ201TICO DE FRECUENCIAS (La magia del sonido)
            switch (perfil)
            {
                case _Scripts.Enums.PerfilAcustico.AgudoVocal:
                    // REGLA AGUDA: Multiplicamos la distancia x 2.0
                    // Si el monstruo estÃ; a 10m, el divisor serÃ; 20. El son
ido se vuelve diminuto muy rÃ;rido.
                    incrementoFinal = volumenOriginal / Mathf.Max(distanciaAlM
onstruo * 2.0f, 0.1f);
                    break;

                case _Scripts.Enums.PerfilAcustico.GraveFisico:
                    // REGLA GRAVE: Multiplicamos la distancia x 0.3 (La reduc
imos)
                    // Si el monstruo estÃ; a 10m, el divisor serÃ; solo 3. La
vibraciÃ³n llega casi intacta.
                    incrementoFinal = volumenOriginal / Mathf.Max(distanciaAlM
onstruo * 0.3f, 0.1f);
                    break;
            }

            // 2. REGLA DEL 75% (Zona de Gracia)
            // Si el monstruo ya estÃ; muy alterado (>0.75), los ruidos le afe
ctan 3 veces menos.
            // AsÃ- le damos al jugador una micro-oportunidad de escapar antes
de pasar a PersecuciÃ³n.
            if (intuicionActual >= 0.75f && incrementoFinal > 0)
            {
                intuicionActual += (incrementoFinal / 3f);
            }
            else
            {
                intuicionActual += incrementoFinal;
            }

            // 3. REGLA DE CASCADA (SUBIDA)
            // Si el vaso se derrama, gritamos por la radio para que el EnemyB
rain suba de fase.
            if (!(intuicionActual >= 1.0f)) return;
            intuicionActual = 0f; // Vaciamos el vaso para el nuevo nivel
            OnSubirFase?.Invoke(); // El '?' asegura que no haya error si nadi
e escucha
        }

        /* * *
         * TICK DE RECUPERACIÃ223N (Llamado por el EnemyBrain cada 60s)
         * Reduce el miedo con el tiempo. Es mÃ;s difÃ-cil calmarlo en niveles
altos.
         */
        public void EjecutarTickDeRecuperacion(int miedoLevel)
        {
            // FÃ223RMULA: ReducciÃ³n = IntuiciÃ³n Actual * (1/3 * 1/Miedo)
            float factorEnfriamiento = (1f / 3f) * (1f / miedoLevel);

```

```

float cantidadARestar = intuicionActual * factorEnfriamiento;

intuicionActual -= cantidadARestar;

// REGLA DE CASCADA (BAJADA)
if (intuicionActual <= 0f)
{
    // REGALO AL JUGADOR: Si el monstruo se calma y baja de fase,
    // la nueva fase empieza en 0.7 (70%) en lugar de 1.0. No se o
lvida de ti del todo.
    intuicionActual = 0.7f;
    OnBajarFase?.Invoke();
}

// Mantenemos el n mero entre 0.0 y 1.0 por seguridad del motor
intuicionActual = Mathf.Clamp01(intuicionActual);
}
}
}
</file>

<file path="Assets/_Scripts/UI/UI_MenuManager.cs">
using _Scripts.Player;
using TMPPro;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.InputSystem;
// LINEA NUEVA: Permite al script tener acceso directo a las herramientas de c
 mara de Cinemachine;
using Cinemachine;

namespace _Scripts.UI
{
    // Asegura que este script inicialice despu s de los componentes base del
    juego para evitar NullReference
    [DefaultExecutionOrder(100)]
    public class UI_MenuManager : MonoBehaviour
    {
        // Enums: Una lista de estados l gicos excluyentes. El juego solo pue
de estar en UNO de estos a la vez.
        private enum MenuState
        {
            Gameplay,
            MenuAbierto
        }

        [Header("Input (Assets/Input/PlayerControls)")]
        [Tooltip("El contenedor de mapas de control (.inputactions) de nuestro
proyecto")]
        [SerializeField]
        private InputActionAsset inputActionAsset;

        [Header("UI")] [Tooltip("El objeto ra z de la interfaz que contiene l
os botones")] [SerializeField]
        private GameObject menuPanel;

        [SerializeField] private TextMeshProUGUI tituloTexto;
        [SerializeField] private TextMeshProUGUI botonPrincipalTexto;

        [Header("Jugador")]

```

```

[Tooltip("Referencia al script que controla el movimiento f sico de n
uestro personaje")]
[SerializeField]
private PlayerController playerController;

[Header("Configuraci n de C mara (Cinemachine)")]
[Tooltip("Arrastra aqu  tu Virtual Camera de Cinemachine para congela
r la rotaci n del mouse en las pausas")]
[SerializeField]
private CinemachineInputProvider cameraProvider;

[SerializeField] private CinemachineVirtualCamera virtualCamera; // <-
- Cambiamos el tipo aqu -
private CinemachinePOV cameraPov; // Ahora es privado, no se arrastra

// Cache de componentes de control para evitar b squeda repetitivas
en cada frame (Ahorro de CPU)
private InputActionMap _gameplayMap;
private InputActionMap _uiMap;
private InputAction _pauseAction;
private InputAction _unpauseAction;

// Estado actual del men  dieg tico (P alterna Gameplay   206\224 Me
nuAbierto).
private MenuState _estado = MenuState.Gameplay;

// Constantes: Textos fijos que no van a cambiar durante la ejecuci n
del c digo

private void Awake()
{
    // Validaci n de seguridad: Si olvidaste arrastrar los controles
en el inspector, el script se apaga solo
    if (inputActionAsset == null)
    {
        Debug.LogError("UI_MenuManager: Asigna el InputActionAsset Pla
yerControls en el Inspector.");
        enabled = false;
        return;
    }

    if (virtualCamera != null)
    {
        // Esto extrae el m dulo interno POV de la c mara virtual
cameraPov = virtualCamera.GetCinemachineComponent<CinemachineP
OV>();

        if (cameraProvider == null)
            cameraProvider = virtualCamera.GetComponent<CinemachineInp
utProvider>();
    }

    // 1. Buscamos y vinculamos los Action Maps definidos en tu archiv
o de inputs
    _gameplayMap = inputActionAsset.FindActionMap("Gameplay", true);
    _uiMap = inputActionAsset.FindActionMap("UI", true);

    // 2. Buscamos las acciones espec ficas mapeadas a la tecla P
_pauseAction = _gameplayMap.FindAction("Pause", true);

```

```

        _unpauseAction = _uiMap.FindAction("Unpause", true);

        // 3. SUSCRIPCIÃ\223N: Conectamos fÃ-sicamente las acciones a nues
tras funciones de C#
        _pauseAction.performed += OnPausePerformed;
        _unpauseAction.performed += OnUnpausePerformed;

        // BÃsqueda automÃtica: Si no arrastraste el script del jugador,
lo intenta buscar en la escena por sÃ mismo
        if (playerController == null)
            playerController = FindObjectOfType<PlayerController>();

        if (cameraProvider == null)
            cameraProvider = FindObjectOfType< CinemachineInputProvider>();

        if (cameraPov == null && cameraProvider != null)
            cameraPov = cameraProvider.GetComponentInChildren< CinemachineP
OV>();
    }

    private void Start()
    {
        if (menuPanel != null)
        {
            Button[] botones = menuPanel.GetComponentsInChildren<Button>(true);
            foreach (Button b in botones)
            {
                if (b.gameObject.name == "Boton_Jugar_Reanudar")
                {
                    b.onClick.RemoveListener(OnClickJugarOReanudar);
                    b.onClick.AddListener(OnClickJugarOReanudar);
                    break;
                }
            }
        }

        EntrarModoGameplay();
    }

    private void OnDestroy()
    {
        // BUENA PRÃ\201CTICA: Rompemos la conexiÃn con los eventos al de
struir el objeto para evitar bugs de memoria
        if (_pauseAction != null)
            _pauseAction.performed -= OnPausePerformed;
        if (_unpauseAction != null)
            _unpauseAction.performed -= OnUnpausePerformed;
    }

    // --- MÃ\211TODOS PÃ\232BLICOS PARA LOS BOTONES (EVENTOS ONCLICK) ---

    public void OnClickJugarOReanudar()
    {
        if (_estado == MenuState.MenuAbierto)
            EntrarModoGameplay();
    }

    public void OnClickSalir()
    {
        // CompilaciÃn condicional: Detiene el juego de forma correcta se

```

```

gÃn el entorno
#if UNITY_EDITOR
    UnityEditor.EditorApplication.isPlaying = false; // Detiene el Pla
y Mode dentro de Unity
#else
    Application.Quit(); // Cierra la aplicaciÃn ejecutable (.exe o bi
nario de Linux)
#endif
}

// --- RESPUESTAS A LOS INPUTS FÃ\215SICOS (TECLA P) ---

private void OnPausePerformed(InputAction.CallbackContext context)
{
    // Filtro de seguridad: Si la acciÃn no se completÃ³ o no estamos
jugando, ignora el teclado
    if (!context.performed || _estado != MenuState.Gameplay)
        return;

    EntrarModoMenuAbierto();
}

private void OnUnpausePerformed(InputAction.CallbackContext context)
{
    if (!context.performed || _estado != MenuState.MenuAbierto)
        return;

    EntrarModoGameplay();
}

// --- MANEJADORES DE ESTADO (MÃ\201QUINA DE ESTADOS) ---

private void EntrarModoGameplay()
{
    _estado = MenuState.Gameplay;

    MostrarMenu(false);
    ConfigurarCursor(false);
    ActivarMapaGameplay();
    ConfigurarJugador(true);
    ConfigurarCamara(true);
}

private void EntrarModoMenuAbierto()
{
    _estado = MenuState.MenuAbierto;

    MostrarMenu(true);
    // ConfigurarTextos(TituloMenu, TextoCerrar); // Eliminado para no sobrees
cribir el diseÃo del usuario
    ConfigurarCursor(true);
    ActivarMapaUI();
    ConfigurarJugador(false);
    ConfigurarCamara(false);
}

// --- SUB-FUNCIONES AUXILIARES DE CONFIGURACIÃ\223N ---

private void ActivarMapaUI()
{

```

```

    _gameplayMap.Disable(); // Desactiva el mapa de juego (WASD deja d
e responder instantáneamente)
    _uiMap.Enable(); // Activa el mapa de UI (Permite que funcione la
tecla Unpause)
}

```

```

private void ActivarMapaGameplay()
{
    _uiMap.Disable(); // Desactiva el mapa de UI
    _gameplayMap.Enable(); // Activa el mapa de juego activo
}

```

```

private void MostrarMenu(bool visible)
{
    if (menuPanel != null)
        menuPanel.SetActive(visible); // Enciende o apaga el GameObjec
t completo en la jerarquía
}

```

```

private void ConfigurarTextos(string titulo, string botonPrincipal)
{
    if (tituloTexto != null)
        tituloTexto.text = titulo;
    if (botonPrincipalTexto != null)
        botonPrincipalTexto.text = botonPrincipal;
}

```

```

private static void ConfigurarCursor(bool menuActivo)
{
    // Si el menú está activo, el cursor se libera. Si no, se bloque
a al centro de la pantalla.
    Cursor.lockState = menuActivo ? CursorLockMode.None : CursorLockMo
de.Locked;
    Cursor.visible = menuActivo; // Alterna la visibilidad del puntero
gráfico
}

```

```

private void ConfigurarJugador(bool activo)
{
    if (playerController != null)
        playerController.SetControlBlocked(!activo);
}

```

```

private void ConfigurarCamara(bool activo)
{
    if (cameraProvider != null)
        cameraProvider.enabled = activo;

    if (cameraPov == null)
        return;

    var horizontal = cameraPov.m_HorizontalAxis;
    horizontal.m_InputAxisValue = 0f;
    cameraPov.m_HorizontalAxis = horizontal;

    var vertical = cameraPov.m_VerticalAxis;
    vertical.m_InputAxisValue = 0f;
    cameraPov.m_VerticalAxis = vertical;

    if (activo && Mouse.current != null)

```

```

        InputSystem.ResetDevice(Mouse.current);
    }
}
</file>

```

```

<file path="Assets/Input/PlayerControls.cs">
//-----
--
// <auto-generated>
// This code was auto-generated by com.unity.inputsystem:InputActionCodeGe
nerator
// version 1.14.2
// from Assets/Input/PlayerControls.inputactions
//
// Changes to this file may cause incorrect behavior and will be lost if
the code is regenerated.
// </auto-generated>
//-----
--

```

```

using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine.InputSystem;
using UnityEngine.InputSystem.Utilities;

```

```

/// <summary>
/// Provides programmatic access to <see cref="InputActionAsset" />, <see cref
="InputActionMap" />, <see cref="InputAction" /> and <see cref="InputControlSc
heme" /> instances defined in asset "Assets/Input/PlayerControls.inputactions"
.
/// </summary>
/// <remarks>
/// This class is source generated and any manual edits will be discarded if t
he associated asset is reimported or modified.
/// </remarks>
/// <example>
/// <code>
/// using namespace UnityEngine;
/// using UnityEngine.InputSystem;
///
/// // Example of using an InputActionMap named "Player" from a UnityEngine.Mo
noBehaviour implementing callback interface.
/// public class Example : MonoBehaviour, MyActions.IPlayerActions
/// {
///     private MyActions.Actions m_Actions; // Source code r
epresentation of asset.
///     private MyActions.Actions.PlayerActions m_Player; // Source code r
epresentation of action map.
///
///     void Awake()
///     {
///         m_Actions = new MyActions.Actions(); // Create asset
object.
///         m_Player = m_Actions.Player; // Extract actio
n map object.
///         m_Player.AddCallbacks(this); // Register call
back interface IPlayerActions.
///     }

```

```

///
/// void OnDestroy()
/// {
///     m_Actions.Dispose();           // Destroy asset
/// }
///
/// void OnEnable()
/// {
///     m_Player.Enable();           // Enable all ac
tions within map.
/// }
///
/// void OnDisable()
/// {
///     m_Player.Disable();           // Disable all a
ctions within map.
/// }
///
/// #region Interface implementation of MyActions.IPlayerActions
///
/// // Invoked when "Move" action is either started, performed or canceled
///
/// public void OnMove(InputAction.CallbackContext context)
/// {
///     Debug.Log($"OnMove: {context.ReadValue<Vector2>();}");
/// }
///
/// // Invoked when "Attack" action is either started, performed or cancel
ed.
/// public void OnAttack(InputAction.CallbackContext context)
/// {
///     Debug.Log($"OnAttack: {context.ReadValue<float>();}");
/// }
///
/// #endregion
/// }
/// </code>
/// </example>
public partial class @PlayerControls: IInputActionCollection2, IDisposable
{
    /// <summary>
    /// Provides access to the underlying asset instance.
    /// </summary>
    public InputActionAsset asset { get; }

    /// <summary>
    /// Constructs a new instance.
    /// </summary>
    public @PlayerControls()
    {
        asset = InputActionAsset.FromJson(@"{
            "version": 1,
            "name": "PlayerControls",
            "maps": [
                {
                    "name": "Gameplay",
                    "id": "7abfd263-bb92-434a-a6e1-384bbda27d16",
                    "actions": [

```

```

                        "name": "Move",
                        "type": "Value",
                        "id": "9c18749d-7a66-4e7c-af1-fd577b715664",
                        "expectedControlType": "Vector2",
                        "processors": "",
                        "interactions": "",
                        "initialStateCheck": true
                    },
                    {
                        "name": "Sprint",
                        "type": "Button",
                        "id": "eb10d26d-9ee2-4ae3-9e82-8874a4c3e951",
                        "expectedControlType": "",
                        "processors": "",
                        "interactions": "",
                        "initialStateCheck": false
                    },
                    {
                        "name": "Crouch",
                        "type": "Button",
                        "id": "9c4689f5-077d-4e7b-a95f-e7990376191c",
                        "expectedControlType": "",
                        "processors": "",
                        "interactions": "",
                        "initialStateCheck": false
                    },
                    {
                        "name": "Pause",
                        "type": "Button",
                        "id": "afeb5324-c85c-4351-bd99-5d6703cdb2af",
                        "expectedControlType": "",
                        "processors": "",
                        "interactions": "",
                        "initialStateCheck": false
                    },
                    {
                        "name": "Look",
                        "type": "Value",
                        "id": "ae16035a-5f2b-41de-a7f3-cb7b3d469034",
                        "expectedControlType": "Vector2",
                        "processors": "",
                        "interactions": "",
                        "initialStateCheck": true
                    }
                ],
                "bindings": [
                    {
                        "name": "2D Vector",
                        "id": "fa8f4d5d-albb-490e-9f42-760d0b8b0da7",
                        "path": "2DVector",
                        "interactions": "",
                        "processors": "",
                        "groups": "",
                        "action": "Move",
                        "isComposite": true,
                        "isPartOfComposite": false
                    },
                    {
                        "name": "up",
                        "id": "a662e232-7817-4fbe-81c8-95daa41d475e",

```

```

        "path": "<Keyboard>/w",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Move",
        "isComposite": false,
        "isPartOfComposite": true
    },
    {
        "name": "down",
        "id": "5114778e-34cb-4c5b-8508-c3b7d0706900",
        "path": "<Keyboard>/s",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Move",
        "isComposite": false,
        "isPartOfComposite": true
    },
    {
        "name": "left",
        "id": "b35f0a8f-0fe1-4b56-9e31-453e95fe1794",
        "path": "<Keyboard>/a",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Move",
        "isComposite": false,
        "isPartOfComposite": true
    },
    {
        "name": "right",
        "id": "405aa2a3-f408-47d6-ba6f-b9d19c015576",
        "path": "<Keyboard>/d",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Move",
        "isComposite": false,
        "isPartOfComposite": true
    },
    {
        "name": "",
        "id": "efc3f80b-b584-4a72-9d00-4cb97d2cd1c9",
        "path": "<XInputController>/leftTrigger",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Sprint",
        "isComposite": false,
        "isPartOfComposite": false
    },
    {
        "name": "",
        "id": "7aff7f1f-414f-47d6-a334-fed5f7e544f",
        "path": "<XInputController>/leftStick/down",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Crouch",

```

```

        "isComposite": false,
        "isPartOfComposite": false
    },
    {
        "name": "",
        "id": "1064827c-1861-4165-ab61-3cdd244c9d32",
        "path": "<Keyboard>/p",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Pause",
        "isComposite": false,
        "isPartOfComposite": false
    },
    {
        "name": "",
        "id": "355006be-f334-47af-834a-315147fdb808",
        "path": "<Mouse>/delta",
        "interactions": "",
        "processors": "",
        "groups": "",
        "action": "Look",
        "isComposite": false,
        "isPartOfComposite": false
    }
]
},
{
    "name": "UI",
    "id": "ca338fad-e951-45db-b139-b0296a2f9796",
    "actions": [
        {
            "name": "Unpause",
            "type": "Button",
            "id": "550a9331-cb76-441c-9cdb-d002470bcb2b",
            "expectedControlType": "",
            "processors": "",
            "interactions": "",
            "initialStateCheck": false
        }
    ],
    "bindings": [
        {
            "name": "",
            "id": "4f82ebaa-9a03-4b18-898a-ba952e7c7bea",
            "path": "<Keyboard>/p",
            "interactions": "",
            "processors": "",
            "groups": "",
            "action": "Unpause",
            "isComposite": false,
            "isPartOfComposite": false
        }
    ]
}
],
"controlSchemes": [
    ];
    // Gameplay
    m_Gameplay = asset.FindActionMap("Gameplay", throwIfNotFound: true);

```

```

        m_Gameplay_Move = m_Gameplay.FindAction("Move", throwIfNotFound: true)
;
        m_Gameplay_Sprint = m_Gameplay.FindAction("Sprint", throwIfNotFound: true);
        m_Gameplay_Crouch = m_Gameplay.FindAction("Crouch", throwIfNotFound: true);
        m_Gameplay_Pause = m_Gameplay.FindAction("Pause", throwIfNotFound: true);
        m_Gameplay_Look = m_Gameplay.FindAction("Look", throwIfNotFound: true)
;
        // UI
        m_UI = asset.FindActionMap("UI", throwIfNotFound: true);
        m_UI_Unpause = m_UI.FindAction("Unpause", throwIfNotFound: true);
    }

    ~@PlayerControls()
    {
        UnityEngine.Debug.Assert(!m_Gameplay.enabled, "This will cause a leak
and performance issues, PlayerControls.Gameplay.Disable() has not been called.
");
        UnityEngine.Debug.Assert(!m_UI.enabled, "This will cause a leak and pe
rformance issues, PlayerControls.UI.Disable() has not been called.");
    }

    /// <summary>
    /// Destroys this asset and all associated <see cref="InputAction"/> insta
nces.
    /// </summary>
    public void Dispose()
    {
        UnityEngine.Object.Destroy(asset);
    }

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.bindingMask" />
    public InputBinding? bindingMask
    {
        get => asset.bindingMask;
        set => asset.bindingMask = value;
    }

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.devices" />
    public ReadOnlyArray<InputDevice>? devices
    {
        get => asset.devices;
        set => asset.devices = value;
    }

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.controlSche
mes" />
    public ReadOnlyArray<InputControlScheme> controlSchemes => asset.controlSc
hemes;

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.Contains(In
putAction)" />
    public bool Contains(InputAction action)
    {
        return asset.Contains(action);
    }

```

```

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.GetEnumerat
or()" />
    public IEnumerator<InputAction> GetEnumerator()
    {
        return asset.GetEnumerator();
    }

    /// <inheritdoc cref="IEnumerable.GetEnumerator()" />
    IEnumerator IEnumerable.GetEnumerator()
    {
        return GetEnumerator();
    }

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.Enable()" /
>
    public void Enable()
    {
        asset.Enable();
    }

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.Disable()" />
    public void Disable()
    {
        asset.Disable();
    }

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.bindings" /
>
    public IEnumerable<InputBinding> bindings => asset.bindings;

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.FindAction(
string, bool)" />
    public InputAction FindAction(string actionNameOrId, bool throwIfNotFound
= false)
    {
        return asset.FindAction(actionNameOrId, throwIfNotFound);
    }

    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionAsset.FindBinding
(InputBinding, out InputAction)" />
    public int FindBinding(InputBinding bindingMask, out InputAction action)
    {
        return asset.FindBinding(bindingMask, out action);
    }

    // Gameplay
    private readonly InputActionMap m_Gameplay;
    private List<IGameplayActions> m_GameplayActionsCallbackInterfaces = new L
ist<IGameplayActions>();
    private readonly InputAction m_Gameplay_Move;
    private readonly InputAction m_Gameplay_Sprint;
    private readonly InputAction m_Gameplay_Crouch;
    private readonly InputAction m_Gameplay_Pause;
    private readonly InputAction m_Gameplay_Look;
    /// <summary>
    /// Provides access to input actions defined in input action map "Gameplay
".
    /// </summary>
    public struct GameplayActions

```



```

{
    private @PlayerControls m_Wrapper;

    /// <summary>
    /// Construct a new instance of the input action map wrapper class.
    /// </summary>
    public GameplayActions(@PlayerControls wrapper) { m_Wrapper = wrapper; }

    /// <summary>
    /// Provides access to the underlying input action "Gameplay/Move".
    /// </summary>
    public InputAction @Move => m_Wrapper.m_Gameplay_Move;
    /// <summary>
    /// Provides access to the underlying input action "Gameplay/Sprint".
    /// </summary>
    public InputAction @Sprint => m_Wrapper.m_Gameplay_Sprint;
    /// <summary>
    /// Provides access to the underlying input action "Gameplay/Crouch".
    /// </summary>
    public InputAction @Crouch => m_Wrapper.m_Gameplay_Crouch;
    /// <summary>
    /// Provides access to the underlying input action "Gameplay/Pause".
    /// </summary>
    public InputAction @Pause => m_Wrapper.m_Gameplay_Pause;
    /// <summary>
    /// Provides access to the underlying input action "Gameplay/Look".
    /// </summary>
    public InputAction @Look => m_Wrapper.m_Gameplay_Look;
    /// <summary>
    /// Provides access to the underlying input action map instance.
    /// </summary>
    public InputActionMap Get() { return m_Wrapper.m_Gameplay; }
    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionMap.Enable()" />
    />

    public void Enable() { Get().Enable(); }
    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionMap.Disable()" />
    " />

    public void Disable() { Get().Disable(); }
    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionMap.enabled" />
    />

    public bool enabled => Get().enabled;
    /// <summary>
    /// Implicitly converts an <see ref="GameplayActions" /> to an <see ref="InputActionMap" /> instance.
    /// </summary>
    public static implicit operator InputActionMap(GameplayActions set) {
return set.Get(); }
    /// <summary>
    /// Adds <see cref="InputAction.started"/>, <see cref="InputAction.performed"/> and <see cref="InputAction.canceled"/> callbacks provided via <param cref="instance" /> on all input actions contained in this map.
    /// </summary>
    /// <param name="instance">Callback instance.</param>
    /// <remarks>
    /// If <paramref name="instance" /> is <c>null</c> or <paramref name="instance"/> have already been added this method does nothing.
    /// </remarks>
    /// <seealso cref="GameplayActions" />
    public void AddCallbacks(IGameplayActions instance)
    {

```

```

        if (instance == null || m_Wrapper.m_GameplayActionsCallbackInterfaces.Contains(instance)) return;
        m_Wrapper.m_GameplayActionsCallbackInterfaces.Add(instance);
        @Move.started += instance.OnMove;
        @Move.performed += instance.OnMove;
        @Move.canceled += instance.OnMove;
        @Sprint.started += instance.OnSprint;
        @Sprint.performed += instance.OnSprint;
        @Sprint.canceled += instance.OnSprint;
        @Crouch.started += instance.OnCrouch;
        @Crouch.performed += instance.OnCrouch;
        @Crouch.canceled += instance.OnCrouch;
        @Pause.started += instance.OnPause;
        @Pause.performed += instance.OnPause;
        @Pause.canceled += instance.OnPause;
        @Look.started += instance.OnLook;
        @Look.performed += instance.OnLook;
        @Look.canceled += instance.OnLook;
    }

    /// <summary>
    /// Removes <see cref="InputAction.started"/>, <see cref="InputAction.performed"/> and <see cref="InputAction.canceled"/> callbacks provided via <param cref="instance" /> on all input actions contained in this map.
    /// </summary>
    /// <remarks>
    /// Calling this method when <paramref name="instance" /> have not previously been registered has no side-effects.
    /// </remarks>
    /// <seealso cref="GameplayActions" />
    private void UnregisterCallbacks(IGameplayActions instance)
    {
        @Move.started -= instance.OnMove;
        @Move.performed -= instance.OnMove;
        @Move.canceled -= instance.OnMove;
        @Sprint.started -= instance.OnSprint;
        @Sprint.performed -= instance.OnSprint;
        @Sprint.canceled -= instance.OnSprint;
        @Crouch.started -= instance.OnCrouch;
        @Crouch.performed -= instance.OnCrouch;
        @Crouch.canceled -= instance.OnCrouch;
        @Pause.started -= instance.OnPause;
        @Pause.performed -= instance.OnPause;
        @Pause.canceled -= instance.OnPause;
        @Look.started -= instance.OnLook;
        @Look.performed -= instance.OnLook;
        @Look.canceled -= instance.OnLook;
    }

    /// <summary>
    /// Unregisters <param cref="instance" /> and unregisters all input action callbacks via <see cref="GameplayActions.UnregisterCallbacks(IGameplayActions)" />.
    /// </summary>
    /// <seealso cref="GameplayActions.UnregisterCallbacks(IGameplayActions)" />
    public void RemoveCallbacks(IGameplayActions instance)
    {
        if (m_Wrapper.m_GameplayActionsCallbackInterfaces.Remove(instance)
)

```

```

        UnregisterCallbacks(instance);
    }

    /// <summary>
    /// Replaces all existing callback instances and previously registered
    input action callbacks associated with them with callbacks provided via <para
m cref="instance" />.
    /// </summary>
    /// <remarks>
    /// If <paramref name="instance" /> is <c>null</c>, calling this metho
d will only unregister all existing callbacks but not register any new callbac
ks.
    /// </remarks>
    /// <seealso cref="GameplayActions.AddCallbacks(IGameplayActions)" />
    /// <seealso cref="GameplayActions.RemoveCallbacks(IGameplayActions)" />
/>

    /// <seealso cref="GameplayActions.UnregisterCallbacks(IGameplayAction
s)" />
    public void SetCallbacks(IGameplayActions instance)
    {
        foreach (var item in m_Wrapper.m_GameplayActionsCallbackInterfaces
)
        {
            UnregisterCallbacks(item);
            m_Wrapper.m_GameplayActionsCallbackInterfaces.Clear();
            AddCallbacks(instance);
        }
    }

    /// <summary>
    /// Provides a new <see cref="GameplayActions" /> instance referencing thi
s action map.
    /// </summary>
    public GameplayActions @Gameplay => new GameplayActions(this);

    // UI
    private readonly InputActionMap m_UI;
    private List<IUIActions> m_UIActionsCallbackInterfaces = new List<IUIActio
ns>();
    private readonly InputAction m_UI_Unpause;
    /// <summary>
    /// Provides access to input actions defined in input action map "UI".
    /// </summary>
    public struct UIActions
    {
        private @PlayerControls m_Wrapper;

        /// <summary>
        /// Construct a new instance of the input action map wrapper class.
        /// </summary>
        public UIActions(@PlayerControls wrapper) { m_Wrapper = wrapper; }
        /// <summary>
        /// Provides access to the underlying input action "UI/Unpause".
        /// </summary>
        public InputAction @Unpause => m_Wrapper.m_UI_Unpause;
        /// <summary>
        /// Provides access to the underlying input action map instance.
        /// </summary>
        public InputActionMap Get() { return m_Wrapper.m_UI; }
        /// <inheritdoc cref="UnityEngine.InputSystem.InputActionMap.Enable()" />
/>

        public void Enable() { Get().Enable(); }

```

```

        /// <inheritdoc cref="UnityEngine.InputSystem.InputActionMap.Disable()" />
    }

    public void Disable() { Get().Disable(); }
    /// <inheritdoc cref="UnityEngine.InputSystem.InputActionMap.enabled" />

    public bool enabled => Get().enabled;
    /// <summary>
    /// Implicitly converts an <see ref="UIActions" /> to an <see ref="Inp
utActionMap" /> instance.
    /// </summary>
    public static implicit operator InputActionMap(UIActions set) { return
set.Get(); }
    /// <summary>
    /// Adds <see cref="InputAction.started"/>, <see cref="InputAction.per
formed"/> and <see cref="InputAction.canceled"/> callbacks provided via <para
m cref="instance" /> on all input actions contained in this map.
    /// </summary>
    /// <param name="instance">Callback instance.</param>
    /// <remarks>
    /// If <paramref name="instance" /> is <c>null</c> or <paramref name="
instance"/> have already been added this method does nothing.
    /// </remarks>
    /// <seealso cref="UIActions" />
    public void AddCallbacks(IUIActions instance)
    {
        if (instance == null || m_Wrapper.m_UIActionsCallbackInterfaces.Co
ntains(instance)) return;
        m_Wrapper.m_UIActionsCallbackInterfaces.Add(instance);
        @Unpause.started += instance.OnUnpause;
        @Unpause.performed += instance.OnUnpause;
        @Unpause.canceled += instance.OnUnpause;
    }

    /// <summary>
    /// Removes <see cref="InputAction.started"/>, <see cref="InputAction.
performed"/> and <see cref="InputAction.canceled"/> callbacks provided via <pa
ram cref="instance" /> on all input actions contained in this map.
    /// </summary>
    /// <remarks>
    /// Calling this method when <paramref name="instance" /> have not pre
viously been registered has no side-effects.
    /// </remarks>
    /// <seealso cref="UIActions" />
    private void UnregisterCallbacks(IUIActions instance)
    {
        @Unpause.started -= instance.OnUnpause;
        @Unpause.performed -= instance.OnUnpause;
        @Unpause.canceled -= instance.OnUnpause;
    }

    /// <summary>
    /// Unregisters <param cref="instance" /> and unregisters all input ac
tion callbacks via <see cref="UIActions.UnregisterCallbacks(IUIActions)" />.
    /// </summary>
    /// <seealso cref="UIActions.UnregisterCallbacks(IUIActions)" />
    public void RemoveCallbacks(IUIActions instance)
    {
        if (m_Wrapper.m_UIActionsCallbackInterfaces.Remove(instance))
            UnregisterCallbacks(instance);
    }

```

```

    /// <summary>
    /// Replaces all existing callback instances and previously registered
    input action callbacks associated with them with callbacks provided via <para
m cref="instance" />.
    /// </summary>
    /// <remarks>
    /// If <paramref name="instance" /> is <c>null</c>, calling this metho
d will only unregister all existing callbacks but not register any new callbac
ks.
    /// </remarks>
    /// <seealso cref="UIActions.AddCallbacks(IUIActions)" />
    /// <seealso cref="UIActions.RemoveCallbacks(IUIActions)" />
    /// <seealso cref="UIActions.UnregisterCallbacks(IUIActions)" />
    public void SetCallbacks(IUIActions instance)
    {
        foreach (var item in m_Wrapper.m_UIActionsCallbackInterfaces)
            UnregisterCallbacks(item);
        m_Wrapper.m_UIActionsCallbackInterfaces.Clear();
        AddCallbacks(instance);
    }
}
/// <summary>
/// Provides a new <see cref="UIActions" /> instance referencing this acti
on map.
/// </summary>
public UIActions @UI => new UIActions(this);
/// <summary>
/// Interface to implement callback methods for all input action callbacks
associated with input actions defined by "Gameplay" which allows adding and r
emoving callbacks.
/// </summary>
/// <seealso cref="GameplayActions.AddCallbacks(IGameplayActions)" />
/// <seealso cref="GameplayActions.RemoveCallbacks(IGameplayActions)" />
public interface IGameplayActions
{
    /// <summary>
    /// Method invoked when associated input action "Move" is either <see
cref="UnityEngine.InputSystem.InputAction.started" />, <see cref="UnityEngine.
InputSystem.InputAction.performed" /> or <see cref="UnityEngine.InputSystem.In
putAction.canceled" />.
    /// </summary>
    /// <seealso cref="UnityEngine.InputSystem.InputAction.started" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.performed" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.canceled" />
    void OnMove(InputAction.CallbackContext context);
    /// <summary>
    /// Method invoked when associated input action "Sprint" is either <se
e cref="UnityEngine.InputSystem.InputAction.started" />, <see cref="UnityEngine.
InputSystem.InputAction.performed" /> or <see cref="UnityEngine.InputSystem.
InputAction.canceled" />.
    /// </summary>
    /// <seealso cref="UnityEngine.InputSystem.InputAction.started" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.performed" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.canceled" />
    void OnSprint(InputAction.CallbackContext context);
    /// <summary>
    /// Method invoked when associated input action "Crouch" is either <se
e cref="UnityEngine.InputSystem.InputAction.started" />, <see cref="UnityEngine.
InputSystem.InputAction.performed" /> or <see cref="UnityEngine.InputSystem.

```

```

InputAction.canceled" />.
    /// </summary>
    /// <seealso cref="UnityEngine.InputSystem.InputAction.started" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.performed" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.canceled" />
    void OnCrouch(InputAction.CallbackContext context);
    /// <summary>
    /// Method invoked when associated input action "Pause" is either <see
cref="UnityEngine.InputSystem.InputAction.started" />, <see cref="UnityEngine.
InputSystem.InputAction.performed" /> or <see cref="UnityEngine.InputSystem.In
putAction.canceled" />.
    /// </summary>
    /// <seealso cref="UnityEngine.InputSystem.InputAction.started" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.performed" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.canceled" />
    void OnPause(InputAction.CallbackContext context);
    /// <summary>
    /// Method invoked when associated input action "Look" is either <see
cref="UnityEngine.InputSystem.InputAction.started" />, <see cref="UnityEngine.
InputSystem.InputAction.performed" /> or <see cref="UnityEngine.InputSystem.In
putAction.canceled" />.
    /// </summary>
    /// <seealso cref="UnityEngine.InputSystem.InputAction.started" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.performed" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.canceled" />
    void OnLook(InputAction.CallbackContext context);
}
/// <summary>
/// Interface to implement callback methods for all input action callbacks
associated with input actions defined by "UI" which allows adding and removin
g callbacks.
/// </summary>
/// <seealso cref="UIActions.AddCallbacks(IUIActions)" />
/// <seealso cref="UIActions.RemoveCallbacks(IUIActions)" />
public interface IUIActions
{
    /// <summary>
    /// Method invoked when associated input action "Unpause" is either <s
ee cref="UnityEngine.InputSystem.InputAction.started" />, <see cref="UnityEngine.
InputSystem.InputAction.performed" /> or <see cref="UnityEngine.InputSystem.
InputAction.canceled" />.
    /// </summary>
    /// <seealso cref="UnityEngine.InputSystem.InputAction.started" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.performed" />
    /// <seealso cref="UnityEngine.InputSystem.InputAction.canceled" />
    void OnUnpause(InputAction.CallbackContext context);
}
}
</file>

<file path="Assets/_Scripts/Brains/EnemyBrain.cs">
using UnityEngine;
using UnityEngine.AI; // Acceso al sistema de navegaci3n
using UnityEngine.SceneManagement; // Importado para reiniciar la escena
using _Scripts.Systems; // Acceso al coraz3n de datos (ScriptableObject)
using _Scripts.Interfaces; // Acceso al contrato IEstado
using _Scripts.States; // Acceso a las l3gicas de estado (Rastreo, Sigi
lo...)

namespace _Scripts.Brain

```

```

{
    /* * * CLASE: EnemyBrain
     * DESCRIPCIÓN: Actúa como el "Cuerpo" y "Contexto" de la IA (Hardware).
     * No decide qué hacer por sí mismo, sino que delega esa tarea a un objeto IEstado.
     * Mantiene vivo el cronómetro de 60s para el enfriamiento.
     * * * * *
     * * */
    public class EnemyBrain : MonoBehaviour
    {
        [Header("Referencias de Datos")]
        [Tooltip("El archivo de datos (SO) que compartimos con el jugador y el sistema")]
        [SerializeField] private IntuicionSystem data;
        public IntuicionSystem Data => data; // Propiedad pública para los Estados

        [Header("Configuración de Movimiento y Navegación")]
        [Tooltip("Agente de navegación del enemigo")]
        [SerializeField] private NavMeshAgent agent;
        public NavMeshAgent Agent => agent; // Propiedad pública para los Estados

        [Tooltip("Velocidad de traslación fñ-sica base.")]
        [SerializeField] private float velocidadBase = 3.5f;

        [Tooltip("Puntos de patrulla para el movimiento perpetuo.")]
        [SerializeField] private Transform[] waypoints;
        public Transform[] Waypoints => waypoints; // Propiedad pública para los Estados

        [Header("Instintos Base")]
        [Tooltip("Transform del jugador para el Olfato")]
        [SerializeField] private Transform transformJugador;
        [Tooltip("Distancia a la que el enemigo mata instantáneamente")]
        [SerializeField] private float distanciaOlfato = 1.5f;

        /* LÓGICA DE MÁQUINA DE ESTADOS e OPTIMIZACIÓN (CACHING) */
        /
        private IEstado _estadoActual;

        // Cacheamos las instancias de los estados para evitar asignaciones de memoria (GC Allocations)
        private IEstado _estadoRastreo;

        [Header("Reloj de Recuperación")]
        // Contador de segundos para llegar al minuto de enfriamiento
        private float _cronometroSesentaSeg = 0f;

        // Guardamos el nivel de fase actual (1, 2, 3...) para pasárselo al S0 en el Tick
        private int _nivelMiedoActual = 1;

        /* * *
         * MÁQUINA TODO: Start
         * Se ejecuta al nacer. Configuramos los estados cacheando referencias y el estado inicial.
         */

```

```

private void Start()
{
    if (!data) return; // Si no hay datos sal de la función.

    // Verificación y asignación automática de componentes obligatorios
    if (!agent) agent = GetComponent<NavMeshAgent>();
    if (!agent) Debug.LogError("EnemyBrain: Falta el componente NavMeshAgent.");
    if (!transformJugador) Debug.LogError("EnemyBrain: Falta asignar el Transform del Jugador para el Olfato.");

    // Suscripción al SO
    data.OnSubirFase += HandleSubirFase;
    data.OnBajarFase += HandleBajarFase;

    // OPTIMIZACIÓN: Instanciamos los estados una sola vez en el inicio
    _estadoRastreo = new EstadoRastreo(this);

    // ESTADO INICIAL: Iniciamos usando la referencia cacheada
    CambiarEstado(_estadoRastreo);
}

/* * *
 * MÁQUINA TODO: Update
 * Ejecución por frame. Mantenemos el pulso de los relojes y la máquina de estados.
 */
private void Update()
{
    if (!data) return;

    // 1. CONDICIÓN GLOBAL (OLFATO): Instinto base primario
    if (transformJugador != null && Vector3.Distance(transform.position, transformJugador.position) <= distanciaOlfato)
    {
        EjecutarMuertePorOlfato();
        return;
    }

    // 2. Manejamos el tiempo para el enfriamiento de 60s
    ManejarCronometroRecuperacion();

    // 3. EJECUCIÓN DEL ESTADO: El estado controla la lógica en este frame
    _estadoActual?.Ejecutar();
}

private void EjecutarMuertePorOlfato()
{
    Debug.Log("<color=red>¡JUGADOR ASESINADO POR OLFATO! Reiniciando escena...</color>");

    // Detenemos el movimiento fñ-sico inmediatamente
    if (agent != null && agent.isOnNavMesh)
    {
        agent.isStopped = true;
    }
}

```

```
// Reiniciamos la escena actual
SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}

/* * *
 * MÃ\211TODO: ActualizarVelocidad
 * FunciÃ³n pÃºblica para que los ESTADOS puedan cambiar la velocidad
del NavMeshAgent.
 */
public void ActualizarVelocidad(float nuevaVelocidad)
{
    velocidadBase = nuevaVelocidad;
    if (agent != null)
    {
        agent.speed = nuevaVelocidad;
    }
}

/* * *
 * MÃ\211TODO: MoverHacia
 * FunciÃ³n pÃºblica para que los ESTADOS ordenen el destino al NavMes
hAgent.
 */
public void MoverHacia(Vector3 destino)
{
    if (agent != null && agent.isOnNavMesh)
    {
        agent.SetDestination(destino);
    }
}

/* * *
 * MÃ\211TODO: CambiarEstado
 * TransiciÃ³n limpia de estados reutilizando referencias existentes.
 */
public void CambiarEstado(IEstado nuevoEstado)
{
    _estadoActual?.Salir();
    _estadoActual = nuevoEstado;
    _estadoActual.Entrar();
}

/* * *
 * MÃ\211TODO: ManejarCronometroRecuperacion
 * Controla el enfriamiento por Ticks.
 */
private void ManejarCronometroRecuperacion()
{
    _cronometroSesentaSeg += Time.deltaTime;

    if (_cronometroSesentaSeg >= 60f)
    {
        data.EjecutarTickDeRecuperacion(_nivelMiedoActual);
        _cronometroSesentaSeg = 0f;
    }
}

// --- MANEJADORES DE EVENTOS ---

private void HandleSubirFase()
```

```
{
    _nivelMiedoActual++;
    Debug.Log($"<color=orange>RADIO: El enemigo subiÃ³ al Nivel {_nive
lMiedoActual}</color>");
    // Nota: AquÃ­ se cambiarÃ¡ al estado correspondiente cacheado en
el futuro
}

private void HandleBajarFase()
{
    _nivelMiedoActual = Mathf.Max(1, _nivelMiedoActual - 1);

    // Si bajamos al nivel base, volvemos a usar la referencia cachead
a de Rastreo
    if (_nivelMiedoActual == 1) CambiarEstado(_estadoRastreo);

    Debug.Log($"<color=cyan>RADIO: El enemigo bajÃ³ al Nivel {_nivelMi
edoActual}</color>");
}

/* * *
 * MÃ\211TODO DE LIMPIEZA: OnDestroy
 * Se ejecuta si matamos al enemigo o cambiamos de escena.
 * Desconecta los cables de radio para no causar errores de memoria.
 */
private void OnDestroy()
{
    if (data != null)
    {
        data.OnSubirFase -= HandleSubirFase;
        data.OnBajarFase -= HandleBajarFase;
    }
}
}
</file>
</files>
```